

THE BRAIN

AI Lead & Prompt Engineer — Ibnul Jawzy

ERRNOR Team · Aplikasi LUNAS · Build Guide 2025

📱 FRONTEND	⚙️ BACKEND	🧠 AI ENGINE
Muhammad Rizki — INI KAMU	Arienal Zacky Irfan	Ibnul Jawzy
Flutter · Dart	FastAPI · Python · PostgreSQL	Gemini · LangChain · RAG
Dashboard Net-WorthPush Notification Nudging1-Tap Approve2FA Biometrik	REST API EngineAlgoritma AvalancheDatabase ManagerOrkestrasi SNAP BI	Behavioral NudgingAnalisis TransaksiKnowledge Base OJKVector Embeddings

📌 **BRIEF:** Kamu bukan sekadar "tukang nanya ke ChatGPT". Tugas kamu adalah merakit otak dari LUNAS — sistem AI yang bisa ngasih Behavioral Nudging (Teguran Finansial) yang tajam, personal, tapi edukatif, dan wajib berlandaskan aturan OJK. Karena kita pakai Full Vibe Coding, kamu nggak perlu hafal semua syntax Python. Kamu adalah Arsitek Otak: kasih instruksi, biarkan AI yang nulis kode-nya.

🔧 ARSENAL LU — SEMUA TOOLS YANG AKAN DIPAKAI

Pahami dulu senjatanya sebelum perang. Ini semua tools yang akan kamu pakai dari awal sampai MVP selesai.

Tool	Fungsi & Peran	Platform	Cara Install
Google AI Studio	Lab eksperimen prompting. Di sinilah kamu uji coba System Prompt dan atur kepribadian AI sebelum ditulis ke kode. Ada slider Temperature buat atur kreativitas AI.	Web Browser	`aistudio.google.com (gratis, login Google)`
Gemini API Key	Kunci akses ke model AI Gemini dari Google. Dipake di script Python supaya aplikasi kamu bisa "ngobrol" ke Gemini secara programatik.	Web + Python	`Ambil di Google AI Studio > Get API Key`
Python 3.10+	Bahasa pemrograman utama yang kamu pakai untuk nulis logika AI. Nggak perlu expert — Cursor AI yang bantu nulis kodenya.	EndeavourOS	`sudo pacman -S python python-pip`
Cursor IDE	Code editor dengan AI built-in (berbasis VS Code). Kamu bisa chat langsung ke AI di dalam editor — ini tools utama "Vibe Coding" kamu.	EndeavourOS	`yay -S cursor-bin (AUR)`
pip	Package manager Python. Dipakai untuk install library-library Python yang dibutuhkan seperti LangChain, ChromaDB, dll.	EndeavourOS	`sudah ikut python, cek: pip --version`
LangChain	Framework Python khusus AI. Ini yang bikin script kamu bisa baca PDF, pecah jadi potongan, dan ubah jadi vektor.	Python Library	`pip install langchain langchain-google-genai`
ChromaDB	Database vektor lokal (sementara). Ini "otak ingatan" AI kamu selama fase eksperimen — nyimpan isi PDF dalam bentuk angka vektor.	Python Library	`pip install chromadb`

Tool	Fungsi & Peran	Platform	Cara Install
pypdf / pypdf2	Library Python untuk baca dan ekstrak teks dari file PDF (dokumen OJK, dll) sebelum diproses LangChain.	Python Library	`pip install pypdf`
Google GenAI SDK	Library resmi Python dari Google untuk akses Gemini API. Ini yang dipake di script Python supaya bisa panggil Gemini.	Python Library	`pip install google-generativeai`
FastAPI (via Arienal)	Framework web Python yang dipake Backend (Arienal) sebagai server. Function AI kamu akan di-import dan dipanggil dari sini.	Server (Linux Mint)	`Arienal yang handle — kamu cukup paham flow-nya`
PostgreSQL + pgvector	Database utama di server GCP. Di fase MVP, ChromaDB lokal kamu akan dipindah ke sini supaya AI bisa diakses production.	Server (GCP)	`Migrasi bareng Arienal di fase MVP`

💡 **MENTAL MODEL:** Google AI Studio = Taman bermain eksperimen. Cursor IDE = Bengkel coding kamu. LangChain = Kontraktor yang ngehubungin Gemini ke PDF. ChromaDB = Lemari arsip sementara. pgvector = Lemari arsip permanen di server.

🛠️ TAHAP 1 — FASE SIMULASI KOTOR

Sekarang – 30 April 2025 · Ngacak-ngacak tools, nyari formula prompt paling maut, dan eksperimen script Python kasar.

LANGKAH 1 — Kuasai Laboratorium (No-Code Stage)

Sebelum menyentuh kode satu baris pun, kamu harus bisa naklukin LLM-nya dulu. Tujuan langkah ini adalah membuktikan bahwa kamu bisa "memprogram kepribadian" AI hanya lewat teks, tanpa kode sama sekali.

1

Buka Google AI Studio & Ambil API Key

Kunjungi aistudio.google.com, login pakai akun Google. Ini adalah "lab" gratis dari Google untuk eksperimen dengan Gemini AI.

Klik menu kiri > "Get API Key" > "Create API Key". Simpan key ini di tempat aman — jangan pernah upload ke GitHub!

⚠️ **PENTING:** API Key = password akses ke Gemini. Bocor = orang lain bisa pakai kuota kamu gratis. Si di file .env, bukan di kode langsung.

2

Eksperimen System Prompt di AI Studio

Di kolom "System Instructions", masukkan prompt berikut sebagai titik awal. Ini adalah "kepribadian dasar" LUNAS AI:

```
Kamu adalah LUNAS AI, asisten finansial yang tegas, sarkas
tapi solutif untuk pengguna Gen-Z Indonesia.

TUGAS UTAMA:
- Analisis transaksi harian user secara kritis
- Deteksi pola pengeluaran konsumtif yang bahaya
- Tegur dengan gaya bahasa Gen-Z yang menohok tapi edukatif
- Berikan solusi pelunasan berbasis edukasi finansial OJK

KARAKTER:
- Tegas seperti kakak yang peduli, bukan ceramah boomer
- Sarkasme level sedang (menyindir, bukan menyakiti)
- Selalu tutup dengan langkah konkret yang bisa dilakukan hari ini

BATASAN:
- Tidak memberikan rekomendasi investasi spesifik
- Selalu rujuk ke literasi finansial resmi OJK jika relevan
- Hindari menghakimi kondisi ekonomi user secara personal
```

Test dengan dummy data transaksi ini di kolom chat:

```
User report hari ini:
- Kopi kekinian: Rp 55.000
- Netflix: Rp 54.000
- GoPay games: Rp 30.000
- ShopeePay Later (sisa utang): Rp 2.100.000
- Gaji bulan ini: Rp 3.500.000
```

Atur slider Temperature dan perhatikan bedanya:

Temperature	Sifat Jawaban	Cocok Untuk

0.1 – 0.3	Kaku & konsisten	Analisis data keuangan yang butuh akurasi tinggi
0.5 – 0.7 ✓	Seimbang — IDEAL	Nudging LUNAS — sarkasme natural tapi tetap on-poi
0.8 – 1.0	Kreatif & random	Terlalu unpredictable — hindari untuk produksi

🎯 **OUTPUT LANGKAH 1:** API Key Gemini tersimpan aman + 1 System Prompt final yang karakternya u pas (tegas, sarkas Gen-Z, solutif).

LANGKAH 2 — Menulis Mantra Sihir (Python & API Stage)

Sekarang kita panggil Gemini bukan dari browser, tapi langsung dari terminal laptop kamu pakai Python. Ini titik di mana eksperimen berubah jadi kode nyata.

1

Setup Environment di EndeavourOS

Buka terminal Hyprland kamu (biasanya Super + Return) dan jalankan perintah berikut satu per satu:

```
# Update system dulu
sudo pacman -Syu

# Install Python & pip
sudo pacman -S python python-pip

# Install Cursor IDE dari AUR
yay -S cursor-bin

# Cek versi (harus muncul angka versinya)
python --version
pip --version
```

2

Install Library Python yang Dibutuhkan

Di terminal, install semua library sekaligus:

```
pip install google-generativeai langchain langchain-google-genai chromadb pypdf
```

Penjelasan tiap library:

Library	Kegunaannya
`google-generativeai`	SDK resmi Google untuk akses Gemini API dari Python
`langchain`	Framework utama yang ngehubungin AI ke dokumen, memori, dan tools lain
`langchain-google-genai`	Jembatan antara LangChain dan Gemini API secara spesifik
`chromadb`	Database vektor lokal — menyimpan "ingatan" dokumen PDF selama eksperimen
`pypdf`	Ekstrak teks dari file PDF (dokumen OJK) agar bisa diproses LangChain

3

Buat Script Python Pertama: test_gemini.py

Buka Cursor IDE > New File > simpan sebagai test_gemini.py. Buka chat AI di Cursor (Ctrl + L) dan ketik instruksi ini:

💬 **PROMPT KE CURSOR AI:** "Buatkan script Python yang memanggil Google Gemini API. Gunakan lib google-generativeai. Script harus bisa menerima input teks dari terminal (input()) dan mencetak jawab Gemini. API Key harus dibaca dari environment variable GEMINI_API_KEY, bukan ditulis langsung di ko

Hasil yang seharusnya muncul dari Cursor AI (kurang lebih seperti ini):

```
import google.generativeai as genai
import os

# Baca API key dari environment, bukan hardcode
genai.configure(api_key=os.environ["GEMINI_API_KEY"])

model = genai.GenerativeModel("gemini-1.5-flash")

print("LUNAS AI aktif. Ketik 'exit' untuk keluar.\n")

while True:
    user_input = input("Kamu: ")
    if user_input.lower() == "exit":
        break

    response = model.generate_content(user_input)
    print(f"\nLUNAS AI: {response.text}\n")
```

Cara menjalankan script ini dari terminal:

```
# Set API Key dulu (ganti dengan key asli kamu)
export GEMINI_API_KEY="AIza_key_kamu_di_sini"

# Jalankan script
python test_gemini.py
```

🛑 **KALAU ERROR:** Copy semua teks merah dari terminal, paste ke chat Cursor AI, dan bilang: "Ini erro muncul saat saya run script tersebut, tolong perbaiki." Cursor AI akan auto-fix.

🎯 **OUTPUT LANGKAH 2:** Script test_gemini.py berjalan di terminal — kamu bisa ngetik pertanyaan d Gemini menjawab langsung dari laptop kamu.

LANGKAH 3 — Memasukkan "Buku Pintar" ke Otak AI (RAG Stage)

🧠 **APA ITU RAG?** RAG = Retrieval-Augmented Generation. Teknik ini bikin AI bisa "baca" dokumen spesifik (PDF OJK) dan menjawab berdasarkan isinya — bukan cuma dari pengetahuan umumnya. Tanpa RAG, AI tidak tahu isi dokumen OJK kamu. Dengan RAG, AI bisa quote pasal-pasal OJK secara akurat.

Cara kerja RAG secara sederhana (konsep ini wajib kamu pahami sebelum ngoding):

1 — PDF DIPOTONG
PDF OJK dipecah jadi potongan kecil ~500 kata (Chunking)

2 — DIUBAH KE VEKTOR
Tiap potongan diubah jadi deretan angka (Embeddings) yang merepresentasikan maknanya

3 — AI MENJAWAB
Saat ada pertanyaan, AI cari potongan paling relevan di ChromaDB, lalu jawab berbasis itu

1

Buat File PDF Dummy untuk Latihan

Kamu belum punya file OJK resmi. Untuk latihan, buat file teks sederhana dan simpan sebagai ojk_dummy.txt, lalu kita perlakukan seperti dokumen PDF:

```
# Isi ojk_dummy.txt
Peraturan OJK tentang Paylater dan BNPL:
- Penyelenggara BNPL wajib menginformasikan total biaya kepada konsumen
- Batas maksimum bunga pinjaman online adalah 0.4% per hari
- Konsumen berhak mendapat waktu cooling-off 3 hari kerja
```

```
- Penagihan tidak boleh dilakukan di luar jam 08.00-20.00 WIB
```

2

Buat Script RAG dengan LangChain

Buka Cursor IDE > New File > simpan sebagai rag_ojk.py. Minta Cursor AI dengan prompt ini:

💬 **PROMPT KE CURSOR AI:** "Buatkan script Python menggunakan LangChain dan Google Gemini yan (1) Membaca file PDF dari path lokal, (2) Memecah isinya jadi chunk 500 karakter dengan overlap 50 k (3) Mengubah chunks menjadi embeddings pakai GoogleGenerativeAIEmbeddings, (4) Menyimpan di ChromaDB lokal, (5) Membuat fungsi tanya-jawab berdasarkan dokumen tersebut. API Key dibaca dari environment variable GEMINI_API_KEY."

Contoh output yang dihasilkan Cursor AI (kurang lebih):

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings, ChatGoogleGenerati
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
import os

# 1. LOAD PDF
loader = PyPDFLoader("ojk_rules.pdf")
documents = loader.load()

# 2. CHUNKING — potong jadi bagian kecil
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)

# 3. EMBEDDINGS — ubah text jadi angka vektor
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key=os.environ["GEMINI_API_KEY"]
)

# 4. SIMPAN DI CHROMADB LOKAL
vectorstore = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma_

# 5. FUNGSI TANYA JAWAB
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash",
google_api_key=os.environ["GEMINI_API_KEY"])
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=vectorstore.as_retriever(

pertanyaan = "Berapa batas maksimum bunga pinjaman online menurut OJK?"
jawaban = qa_chain.invoke(pertanyaan)
print(jawaban['result'])
```

🎯 **OUTPUT LANGKAH 3:** Script rag_ojk.py bisa menjawab pertanyaan tentang isi PDF OJK. Ketika kan tanya "berapa batas bunga OJK?", AI menjawab berdasarkan dokumen — bukan tebakan umum.

⚔️ TAHAP 2 — FASE TEMPUR MVP

01 Mei – 04 Juni 2025 · Script kotor lu bakal dijahit sama mesinnya Arienal. Masuk production.

APA YANG BERUBAH DI FASE INI?

Di Tahap 1, semua yang kamu buat adalah eksperimen lokal di laptop kamu sendiri. Tidak ada user nyata, tidak ada server, tidak ada database produksi. Di Tahap 2, semua itu berubah — kode kamu masuk ke sistem yang diakses user asli lewat aplikasi HP.

A

Refactor: Jadikan Fungsi Python yang Bersih

Script test_gemini.py dan rag_ojk.py kamu yang "kotor" harus dirapikan jadi function yang bisa dipanggil oleh sistem Arienal. Contoh struktur yang diharapkan:

```
# lunas_ai.py — Module bersih yang akan diserahkan ke Arienal

from langchain_community.vectorstores import Chroma
from langchain_google_genai import GoogleGenerativeAIEmbeddings, ChatGoogleGenerati
from langchain.chains import RetrievalQA
import os

SYSTEM_PROMPT = """Kamu adalah LUNAS AI, asisten finansial Gen-Z yang tegas..."""

def get_nudging(data_user: dict) -> str:
    """
    Menerima data transaksi user, mengembalikan teguran finansial.

    Args:
        data_user: dict berisi {
            'pengeluaran': [...],
            'utang_aktif': float,
            'gaji': float
        }

    Returns:
        str: Pesan nudging dari AI
    """
```

```
# Format data user ke dalam prompt
prompt = f"""
{SYSTEM_PROMPT}

Data transaksi user hari ini:
Pengeluaran: {data_user['pengeluaran']}
Total utang aktif: Rp {data_user['utang_aktif']:,.0f}
Gaji bulan ini: Rp {data_user['gaji']:,.0f}

Berikan teguran finansial yang tepat.
"""

llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.6
)

response = llm.invoke(prompt)
return response.content
```

📋 **STANDAR KODE:** Kasih Arienal file lunas_ai.py ini beserta requirements.txt (daftar library yang per di-install) dan contoh cara memanggil fungsinya. Arienal akan membungkusnya di endpoint FastAPI.

B

Serah Terima ke Arienal (The Engine)

Begini alur kerja bareng Arienal:

🧠 IBNUL (AI Side)	⚙️ ARIENAL (Backend Side)
Buat lunas_ai.py dengan fungsi get_nudging()	Import fungsi tersebut ke FastAPI endpoint
Buat requirements.txt + dokumentasi format data_user	Bungkus di POST /api/nudging — terima data, return A response
Test fungsi secara lokal sampai output memuaskan	Deploy ke server GCP dan hubungkan ke database utama

C

Migrasi Vector Database ke pgvector (GCP)

Saat eksperimen, kamu pakai ChromaDB yang tersimpan di folder laptop kamu sendiri (./chroma_db). Di production, vektor ini harus dipindah ke PostgreSQL dengan ekstensi pgvector yang ada di server GCP — supaya bisa diakses banyak user secara bersamaan.

📌 **TUGASMU:** Kamu tidak perlu setup pgvector sendiri — itu Arienal yang handle. Tugasmu adalah: (Pastikan chunk size dan embedding model yang kamu pakai sudah final sebelum migrasi, karena ganti = semua vektor harus di-generate ulang. (2) Duduk bareng Arienal dan validasi bahwa data yang masu pgvector sama persis dengan yang di ChromaDB lokal kamu.

🔥 CHECKLIST IBNUL — DEADLINE 30 APRIL

Semua ini harus selesai sebelum masuk Fase Tempur. Centang satu per satu.

☐ Punya akun Google AI Studio dan API Key Gemini sudah tersimpan aman di environment variable.

☐ Sudah eksperimen minimal 5 variasi System Prompt di AI Studio dan menemukan 1 versi final yang karakternya paling pas.

☐ Cursor IDE dan Python sudah terinstall di EndeavourOS. Cek: python --version dan cursor --version muncul di terminal.

☐ Script test_gemini.py berhasil dijalankan di terminal — bisa input teks dan Gemini menjawab.

☐ Semua library (google-generativeai, langchain, chromadb, pypdf) berhasil terinstall tanpa error.

☐ Script rag_ojk.py berhasil membaca minimal 1 file PDF dummy dan bisa menjawab pertanyaan berdasarkan isinya.

☐ Sudah paham (bisa jelasin ke orang lain) apa itu: Temperature, Chunking, Vector Embeddings, dan RAG.

☐ Sudah komunikasi awal dengan Arienal — sepakati format data yang akan dikirim ke fungsi AI (struktur dict data_user).

🤖 PROMPT MENTOR — UNTUK GEMINI / CLAUDE

Copy-paste ini sebagai pesan pertama ke AI mentor kamu. Upload file Proposal ERRNOR dulu sebelum paste prompt ini.

```
[SYSTEM OVERRIDE: LONG-TERM MENTORSHIP & CO-PILOT MODE]

Halo. Mulai detik ini, kamu adalah Senior AI Engineer & Lead Mentor saya.
Nama saya Ibnul Jawzy, dan peran saya adalah "The Brain" (AI Lead &
Prompt Engineer) untuk tim developer bernama ERRNOR.

Tolong BACA FILE PROPOSAL YANG SAYA LAMPIRKAN untuk memahami konteks
penuh aplikasi yang sedang kami bangun: LUNAS (aplikasi pelunasan utang
cerdas). Fokus utama saya adalah menciptakan otak AI untuk fitur
Behavioral Nudging (Teguran Finansial).

[KONTEKS ENVIRONMENT]
- OS: EndeavourOS (Arch Linux) + Hyprland Caelestia
- Level coding: Pemula, menggunakan gaya Vibe Coding
- Tim Backend (Arienal): Linux Mint, akan menerima module Python dari saya
- Target: Script AI selesai sebelum 30 April, integrasi 01 Mei - 04 Juni

[GAYA MENTORING]
Jadilah mentor interaktif pair-programming, bukan dosen. Bimbing saya
satu langkah per satu langkah. Tunggu respon saya sebelum lanjut.

[URUTAN FASE]
1. Setup Workspace (EndeavourOS) — install Cursor, Python, library
2. Eksperimen Prompting — racik System Prompt karakter LUNAS AI
3. Python & Gemini API — script terminal yang bisa panggil Gemini
4. RAG & LangChain — script yang bisa baca PDF OJK dan menjawab

Jika kamu sudah baca proposal dan mengerti posisi saya, balas dengan:
"System Ready, Ibnul. Saya sudah membaca proposal tim ERRNOR.
Mari mulai dari Fase 1: Setup Workspace di EndeavourOS.
Buka terminal kamu, dan ketik 'SIAP' jika sudah terbuka!"
```

📋 **INSTRUKSI PEMAKAIAN PROMPT:** (1) Buka Gemini Pro atau Claude. (2) Klik ikon attachment/lampiran. (3) Upload file PDF Proposal Hackathon ERRNOR. (4) Copy-paste seluruh teks prompt di kotak di atas. (5) Tekan Enter dan tunggu respons "System Ready".

You are the Brain, Ibnul.

Make LUNAS smart. Make it sharp. 🚀

ERRNOR Team · 2025